In [1]:
```
%pip install -q "huggingface_hub[hf_xet]"
```

Note: you may need to restart the kernel to use updated packages.

In [3]:
```
# ===== Master v3 — Cell 1: Imports & Key Detection (fixed) =====
import os, re, ast, json, pickle, warnings
from pathlib import Path
import numpy as np, pandas as pd
warnings.filterwarnings("ignore")

from sentence_transformers import SentenceTransformer
import faiss
from tensorflow import keras

ART = Path(r"C:\Users\kylek\artifacts").resolve()
ART.mkdir(parents=True, exist_ok=True)
print("ART set to:", ART)

#assert (ART/"faiss_index.bin").exists(), "Missing FAISS index"
#assert (ART/"movie_embeddings.npy").exists(), "Missing embeddings"
#assert (ART/"movie_ids.npy").exists(), "Missing movie_ids.npy"
#assert (ART/"svd_model.pkl").exists() and (ART/"ncf_model.keras").exists(), "Missi

# Load embeddings & FAISS
#emb = np.load(ART/"movie_embeddings.npy")
#emb_ids = np.load(ART/"movie_ids.npy", allow_pickle=True).tolist()
#faiss_index = faiss.read_index(str(ART/"faiss_index.bin"))

def norm_title(s: str) -> str:
    s = (str(s) or "").lower()
    s = re.sub(r"[^a-z0-9\s]", " ", s)   # <-- fixed: pass 's' as third arg
    return " ".join(s.split())

#def is_intable(x):
    #try:
        #int(str(x))
        #return True
    #except Exception:
        #return False

#numeric_count = sum(is_intable(x) for x in emb_ids)
#TITLE_MODE = numeric_count < 0.5 * len(emb_ids)

#emb_keys = [norm_title(x) for x in emb_ids] if TITLE_MODE else [int(str(x)) for x
#key2idx = {k: i for i, k in enumerate(emb_keys)}
#key_set = set(key2idx.keys())

#print(f"✅ Emb: {emb.shape}, FAISS={faiss_index.ntotal}, key_mode={'title' if TITL
```

```
ART set to: C:\Users\kylek\artifacts
```

In [5]:
```
# ===== Cell 2: CF models & ID maps =====
import json, pickle
from tensorflow import keras
```

```
with open(ART/"svd_model.pkl","rb") as f:
    svd_model = pickle.load(f)

ncf_model = keras.models.load_model(ART/"ncf_model.keras")

with open(ART/"user_to_idx.json") as f:
    user_to_idx = json.load(f)

with open(ART/"movie_to_idx.json") as f:
    movie_to_idx = json.load(f)

print(f"✅ CF loaded | users={len(user_to_idx):,} movies={len(movie_to_idx):,}")
```

```
✅ CF loaded | users=330,712 movies=97,170
```

In [7]:

```
# ===== Cell 3: TMDB + Themes + Sentiments + WR + lead_gender =====
import ast, numpy as np, pickle
import pandas as pd

# Load TMDB
tmdb = pd.read_parquet("tmdb_fully_enriched.parquet")
tmdb["title_norm"] = tmdb["tmdb_title"].apply(norm_title)
print(f"✅ TMDB loaded: {len(tmdb):,} rows")

# Robust genre names
GENRE_MAPPING = {
    28:"Action",12:"Adventure",16:"Animation",35:"Comedy",80:"Crime",99:"Documentar
    18:"Drama",10751:"Family",14:"Fantasy",36:"History",27:"Horror",10402:"Music",
    9648:"Mystery",10749:"Romance",878:"Science Fiction",10770:"TV Movie",53:"Thril
    10752:"War",37:"Western"
}
def to_genre_names(g):
    if isinstance(g,(list,np.ndarray)):
        ids=list(g)
    elif isinstance(g,str):
        try:
            p=ast.literal_eval(g)
            ids=list(p) if isinstance(p,(list,tuple,np.ndarray)) else []
        except Exception:
            ids=[]
    else:
        ids=[]
    out=[]
    for x in ids:
        try:
            name=GENRE_MAPPING.get(int(x))
            if name: out.append(name)
        except Exception:
            pass
    return out

tmdb["genre_names"] = tmdb["genre_ids"].apply(to_genre_names)
print("🎭 genre_names non-empty:", (tmdb["genre_names"].str.len()>0).sum())

# Weighted rating (IMDB-style)
if "weighted_rating" not in tmdb.columns:
```

```
    C = tmdb["vote_average"].mean()
    m = 1000
    def wr(row, C=C, m=m):
        v = float(row.get("vote_count",0) or 0)
        R = float(row.get("vote_average",C) or C)
        return (v/(v+m))*R + (m/(v+m))*C
    tmdb["weighted_rating"] = tmdb.apply(wr, axis=1)
print("⚖️ weighted_rating ready; sample:", round(float(tmdb["weighted_rating"].iloc

# Themes (LDA over RT)
with open(ART/"movie_themes.pkl","rb") as f:
    theme_art = pickle.load(f)
theme_df = pd.DataFrame({
    "title_norm": [norm_title(t) for t in theme_art["movie_titles"]],
    "lda_themes": theme_art["themes"]
})
tmdb = tmdb.merge(theme_df, on="title_norm", how="left")
tmdb["lda_themes"] = tmdb["lda_themes"].apply(
    lambda x: x if isinstance(x, list) else ([] if pd.isna(x) else [x])
)
print("🎭 themes merged:", tmdb["lda_themes"].map(len).gt(0).sum())

# Sentiments (movie-level BERT aggregation)
sent = pd.read_pickle(ART/"movie_sentiments.pkl")
sent["title_norm"] = sent["movie_title"].apply(norm_title)
emo_cols = [c for c in sent.columns if c not in ["movie_title","rotten_tomatoes_lin
tmdb = tmdb.merge(sent[["title_norm"]+emo_cols], on="title_norm", how="left")
print("🧠 sentiments merged (happy non-null):", tmdb["sentiment_happy"].notna().sum

# lead_gender from top-billed cast if available
import ast

def _as_list(x):
    if isinstance(x, list): return x
    if isinstance(x, str):
        try: return ast.literal_eval(x)
        except Exception: return []
    return x if isinstance(x, list) else []

def lead_gender(row):
    c = _as_list(row.get("cast", []))
    if isinstance(c, list) and c and isinstance(c[0], dict):
        g = c[0].get("gender")
        return "female" if g == 1 else ("male" if g == 2 else None)
    return None

tmdb["lead_gender"] = tmdb.apply(lead_gender, axis=1)
print("lead_gender non-null:", tmdb["lead_gender"].notna().sum())

# Quick preview
sample = tmdb.iloc[0]
print(f"🔎 sample: {sample['tmdb_title']} ({sample.get('year')}) | WR={round(float
```

```
✅ TMDB loaded: 43,858 rows
🎭 genre_names non-empty: 39985
⚖️ weighted_rating ready; sample: 6.06
🎭 themes merged: 16259
🧠 sentiments merged (happy non-null): 17903
lead_gender non-null: 0
🔎 sample: Percy Jackson & the Olympians: The Lightning Thief (2010) | WR=6.06
```

In [9]:

```python
# === Rebuild FAISS on current TMDB catalog (aligned keys) ===
import numpy as np, faiss
from sentence_transformers import SentenceTransformer

# 1) Keys aligned to TMDB
tmdb = tmdb.copy()
tmdb["__key__"] = tmdb["title_norm"].astype(str)

# 2) Embed Title + Overview + Genres
def _concat_text(row):
    g = " ".join(row.get("genre_names", [])) if isinstance(row.get("genre_names"),
    return f"{row.get('tmdb_title','')} {row.get('overview','')} {g}".strip()

texts = tmdb.apply(_concat_text, axis=1).fillna("").str.slice(0, 700).tolist()

# 3) Encode & normalize (cosine similarity via inner product)
encoder = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")
emb = encoder.encode(
    texts, batch_size=64, show_progress_bar=True, normalize_embeddings=True
).astype("float32")

# 4) Build FAISS index
d = emb.shape[1]
faiss_index = faiss.IndexFlatIP(d)
faiss_index.add(emb)

# 5) Lookup structures aligned with TMDB
emb_keys = tmdb["__key__"].tolist()
key2idx = {k: i for i, k in enumerate(emb_keys)}
key_set = set(emb_keys)

print(f"✅ FAISS rebuilt on TMDB: {faiss_index.ntotal} vectors; tmdb={len(tmdb)}")
```

```
Batches:   0%|          | 0/723 [00:00<?, ?it/s]
✅ FAISS rebuilt on TMDB: 46229 vectors; tmdb=46229
```

In [10]:

```python
overlap = len(set(emb_keys) & set(tmdb["title_norm"])) / max(1, len(tmdb))
print(f"Key overlap with TMDB (~100% expected): {overlap:.1%}")
```

```
Key overlap with TMDB (~100% expected): 90.1%
```

In [11]:

```python
# === Normalize `cast` to list[str] for reliable actor filtering ===
import ast

def _cast_to_list(v):
    # Already a list
    if isinstance(v, list):
        return [str(x).strip() for x in v if str(x).strip()]
    # String forms: JSON-like ["A","B"], pipe- or comma-separated, or single name
```

```
    if isinstance(v, str):
        s = v.strip()
        if not s:
            return []
        # JSON/list-as-string
        if (s.startswith("[") and s.endswith("]")) or (s.startswith("(") and s.ends
            try:
                parsed = ast.literal_eval(s)
                if isinstance(parsed, (list, tuple)):
                    return [str(x).strip() for x in parsed if str(x).strip()]
            except Exception:
                pass
        # Pipe or comma separated fallback
        if "|" in s:
            return [x.strip() for x in s.split("|") if x.strip()]
        if "," in s:
            return [x.strip() for x in s.split(",") if x.strip()]
        # Single name as last resort
        return [s]
    # Anything else (NaN, None, etc.)
    return []

if "cast" in tmdb.columns:
    tmdb["cast"] = tmdb["cast"].apply(_cast_to_list)
else:
    print("⚠️ `cast` column not found in TMDB; actor filtering will be skipped.")

# quick sanity
print("✅ cast normalized:",
      tmdb["cast"].map(lambda x: isinstance(x, list)).mean(),
      " (fraction of rows with list)")
```

```
✅ cast normalized: 1.0  (fraction of rows with list)
```

In [12]:

```
# Should be > 0 for well-known actors
tmdb[tmdb["cast"].apply(lambda L: any("bill murray" in n.lower() for n in L))].head
```

Out[12]:

tmdb_title	year	cast

In [13]:

```
# ===== UNIFIED Zero-Shot Tagging — Complete Label Set =====
# Tags ALL movies with: emotional + subject + character + relationship + theme labe
# Optimized for multi-core processing, resumable sharding
# Saves to: zs_unified_checkpoint.parquet (new file, won't overwrite old checkpoint

import os
import pandas as pd
from tqdm import tqdm
from pathlib import Path
import time
import re

# Force Transformers to ignore TF/Keras
os.environ["TRANSFORMERS_NO_TF"] = "1"
os.environ["HF_HUB_DISABLE_TELEMETRY"] = "1"
```

```python
# Ensure PyTorch is available
try:
    import torch
except Exception as e:
    raise RuntimeError("PyTorch is not installed. Install a CPU build of torch and

print("="*80)
print("UNIFIED ZERO-SHOT TAGGING — COMPLETE LABEL SET")
print("="*80)

# --- Build zero-shot classifier (PyTorch backend ONLY)
print("\n🔧 Initializing zero-shot classifier...")
from transformers import pipeline

classifier = pipeline(
    task="zero-shot-classification",
    model="facebook/bart-large-mnli",
    framework="pt",  # PyTorch only
    device=-1        # CPU (set to 0 for CUDA GPU if you have one)
)
print("✅ Classifier loaded: facebook/bart-large-mnli\n")

# --- Define norm_title function
def norm_title(s: str) -> str:
    s = (str(s) or "").lower()
    s = re.sub(r"[^a-z0-9\s]", " ", s)
    return " ".join(s.split())

# --- COMPLETE UNIFIED LABEL SET (~80 labels) ---
UNIFIED_LABELS = [
    # === EMOTIONAL / TONAL (from Cell 3.5) ===
    "inspiring", "uplifting", "comforting", "feel-good", "family friendly",
    "family values", "tragic", "heartbreaking", "bittersweet", "lost love",
    "romantic", "slow-burn", "dark", "bleak",
    "mystery", "suspenseful", "thriller", "documentary", "coming of age",
    "redemption", "friendship", "found family", "biographical",

    # === SUBJECT / GENRE (from Cell 3.6) ===
    # Crime / thriller / espionage
    "mafia", "organized crime", "gangster", "mob", "crime", "noir",
    "heist", "detective", "police procedural", "courtroom", "espionage",
    "political thriller",

    # Sci-fi / speculative
    "sci-fi", "science fiction", "cyberpunk", "dystopian", "post-apocalyptic",
    "time travel", "space opera", "alien", "robot ai", "kaiju",

    # Horror / creatures
    "vampire", "zombie", "psychological horror",

    # Action / martial / western
    "superhero", "martial arts", "western",

    # Drama / art
    "period drama", "historical epic", "surreal", "absurdist", "existential",
    "arthouse", "satire", "parody",
```

```
    # Fantasy
    "fantasy", "sword and sorcery", "myth and legend",

    # === CHARACTER TYPES (NEW) ===
    "strong female lead", "female protagonist", "male protagonist",
    "ensemble cast", "anti-hero", "underdog story", "mentor relationship",
    "father-son relationship", "mother-daughter relationship", "unlikely friendship

    # === RELATIONSHIP DYNAMICS (NEW) ===
    "forbidden love", "unrequited love", "toxic relationship", "family drama",
    "generational conflict", "class struggle", "cultural clash",

    # === EMOTIONAL JOURNEY / THEMES (NEW) ===
    "revenge", "betrayal", "sacrifice", "survival", "overcoming adversity",
    "self-discovery", "identity crisis", "moral dilemma", "loss and grief",
    "hope", "justice vs revenge", "loneliness", "power and corruption",

    # === PACING / ATMOSPHERE (NEW) ===
    "fast-paced", "intense", "contemplative", "whimsical", "gritty",
    "atmospheric", "cerebral",

    # === SETTING / SCOPE (NEW) ===
    "small-town", "isolated setting", "road trip", "epic scope", "intimate story"
]

print(f"📋 Total labels in unified set: {len(UNIFIED_LABELS)}")
print(f"   Emotional/Tonal: ~23")
print(f"   Subject/Genre: ~33")
print(f"   Character Types: ~10")
print(f"   Relationships: ~7")
print(f"   Themes/Journey: ~13")
print(f"   Pacing/Setting: ~11\n")

# --- Tagging function (optimized)
MAX_CHARS = 700  # Sweet spot for speed vs quality

def unified_zs_tags(text: str, top_k=3, conf=0.30):
    """
    Tags text with unified label set.
    Uses first 350 + last 350 chars to capture setup and resolution.
    """
    if not isinstance(text, str) or len(text) < 40:
        return []

    # Smart truncation: first half + last half
    if len(text) > MAX_CHARS:
        mid = MAX_CHARS // 2
        txt = text[:mid] + " " + text[-mid:]
    else:
        txt = text

    out = classifier(txt, candidate_labels=UNIFIED_LABELS, multi_label=True)
    scored = sorted(zip(out["labels"], out["scores"]), key=lambda x: x[1], reverse=
    return [lbl for lbl, s in scored[:top_k] if s >= conf]
```

```python
# --- Load and aggregate review text ---
print("📚 Loading reviews...")
reviews_path = ART / "reviews_with_emotions.parquet"
assert reviews_path.exists(), f"Missing {reviews_path}"

reviews_df = pd.read_parquet(reviews_path)
reviews_df["title_norm"] = reviews_df["movie_title"].apply(norm_title)

print(f"   Loaded {len(reviews_df):,} reviews")

# Aggregate review text per movie
agg = (
    reviews_df.groupby("title_norm", as_index=False)["review_content"]
    .agg(lambda x: " ".join(map(str, x)))
)

print(f"   Aggregated into {len(agg):,} movies")
print(f"   Avg review length: {agg['review_content'].str.len().mean():.0f} chars\n"

# --- Setup unified checkpoint directory ---
unified_shard_dir = ART / "zs_unified_shards"
unified_shard_dir.mkdir(parents=True, exist_ok=True)

# Load already-done titles from shards
done_titles = set()
shard_count = 0

for p in unified_shard_dir.glob("unified_shard_*.parquet"):
    try:
        dfp = pd.read_parquet(p, columns=["title_norm"])
        done_titles.update(dfp["title_norm"].tolist())
        shard_count += 1
    except Exception:
        pass

if shard_count > 0:
    print(f"♻️  Found {shard_count} existing shards")
    print(f"✅ Already tagged: {len(done_titles):,} movies\n")

# Filter out done titles
todo = agg[~agg["title_norm"].isin(done_titles)].reset_index(drop=True)

print(f"🎯 TAGGING PLAN:")
print(f"   Total movies: {len(agg):,}")
print(f"   Already done: {len(done_titles):,}")
print(f"   Remaining: {len(todo):,}")
print(f"   Est. time: ~{len(todo) * 4.5 / 3600:.1f} hours (at 4.5 sec/movie)\n")

if len(todo) == 0:
    print("✅ All movies already tagged!")
else:
    # --- Tag in batches with progress tracking ---
    BATCH_SIZE = 32  # Optimized for your 12-core Ryzen 9

    shard_num = shard_count  # Continue numbering from existing shards
```

```
    print(f"🏷️  Starting unified tagging...")
    print(f"   Batch size: {BATCH_SIZE} movies per shard")
    print(f"   Total batches: {len(todo) // BATCH_SIZE + 1}\n")

    start_time = time.time()
    batch_times = []

    for i in tqdm(range(0, len(todo), BATCH_SIZE), desc="Unified tagging"):
        batch_start = time.time()

        chunk = todo.iloc[i:i+BATCH_SIZE]

        # Tag each title in chunk
        tags = [unified_zs_tags(t) for t in chunk["review_content"].tolist()]

        # Create shard dataframe
        part = pd.DataFrame({
            "title_norm": chunk["title_norm"].tolist(),
            "unified_tags": tags
        })

        # Write shard
        shard_path = unified_shard_dir / f"unified_shard_{shard_num:06d}.parquet"
        part.to_parquet(shard_path, index=False)

        shard_num += 1
        batch_times.append(time.time() - batch_start)

        # Progress update every 25 shards
        if shard_num % 25 == 0:
            elapsed = time.time() - start_time
            titles_done = min(i + BATCH_SIZE, len(todo))

            # Calculate rate from recent batches
            recent_rate = sum(batch_times[-25:]) / len(batch_times[-25:]) / BATCH_S
            remaining_titles = len(todo) - titles_done
            remaining_time = remaining_titles * recent_rate

            print(f"\n  ✓ Shard {shard_num:06d} | "
                  f"{titles_done:,}/{len(todo):,} titles ({titles_done/len(todo)*10
                  f"{1/recent_rate:.1f} titles/sec | "
                  f"~{remaining_time/60:.0f}min remaining")

    total_time = time.time() - start_time
    print(f"\n✅ Tagging complete!")
    print(f"   Time: {total_time/60:.1f} minutes ({total_time/3600:.2f} hours)")
    print(f"   Average: {total_time/len(todo):.2f} sec/movie")
    print(f"   New shards written: {shard_num - shard_count}")

# --- CONSOLIDATE ALL SHARDS INTO MASTER CHECKPOINT ---
print("\n" + "="*80)
print("📦 CONSOLIDATING SHARDS...")
shards = list(unified_shard_dir.glob("unified_shard_*.parquet"))

if shards:
    print(f"Loading {len(shards)} shard files...")
```

```
    unified_df = (
        pd.concat([pd.read_parquet(p) for p in shards], ignore_index=True)
          .drop_duplicates("title_norm", keep="last")
    )

    # --- Normalize unified_tags to real Python lists (prevents 0-tag reporting) --
    import numpy as np, ast
    def to_pylist(x):
        if isinstance(x, list): return x
        if isinstance(x, np.ndarray): return x.tolist()
        if x is None: return []
        if isinstance(x, str):
            try: return ast.literal_eval(x)
            except Exception: return [x]
        try: return list(x)
        except Exception: return []

    unified_df["unified_tags"] = unified_df["unified_tags"].apply(to_pylist)
    print(f"✅ Loaded {len(unified_df):,} unique movies")

    # --- Save master checkpoint ---
    master_checkpoint = ART / "zs_unified_checkpoint.parquet"
    unified_df.to_parquet(master_checkpoint, index=False)
    print(f"💾 Saved master checkpoint: {master_checkpoint.name}")

else:
    print("⚠️ No shards found - nothing to consolidate")
```

```
================================================================================
UNIFIED ZERO-SHOT TAGGING — COMPLETE LABEL SET
================================================================================

🔧 Initializing zero-shot classifier...
Device set to use cpu
```

```
✅ Classifier loaded: facebook/bart-large-mnli

📋 Total labels in unified set: 104
   Emotional/Tonal: ~23
   Subject/Genre: ~33
   Character Types: ~10
   Relationships: ~7
   Themes/Journey: ~13
   Pacing/Setting: ~11

📚 Loading reviews...
   Loaded 976,093 reviews
   Aggregated into 17,023 movies
   Avg review length: 7878 chars

♻️  Found 532 existing shards
✅ Already tagged: 17,023 movies

🎯 TAGGING PLAN:
   Total movies: 17,023
   Already done: 17,023
   Remaining: 0
   Est. time: ~0.0 hours (at 4.5 sec/movie)

✅ All movies already tagged!

================================================================================
📦 CONSOLIDATING SHARDS...
Loading 532 shard files...
✅ Loaded 17,023 unique movies
💾 Saved master checkpoint: zs_unified_checkpoint.parquet
```

In [19]:

```python
# ===== Cell 26: Merge V2 Zero-Shot Tags (Additional Labels) =====
# Same logic as V1 merge (Cell 25), but for the V2 checkpoint

import pandas as pd
import numpy as np
import ast
from pathlib import Path

print("="*80)
print("📦 CONSOLIDATING V2 ZERO-SHOT TAGS")
print("="*80)

# Load V2 checkpoint
master_checkpoint_v2 = ART / "zs_unified_checkpoint_v2.parquet"
assert master_checkpoint_v2.exists(), f"Missing {master_checkpoint_v2}"

unified_df_v2 = pd.read_parquet(master_checkpoint_v2)

# Use same normalization function as V1
def to_pylist(x):
    if isinstance(x, list): return x
    if isinstance(x, np.ndarray): return x.tolist()
    if x is None: return []
    if isinstance(x, str):
```

```
        try: return ast.literal_eval(x)
        except Exception: return [x]
    try: return list(x)
    except Exception: return []

# Normalize V2 tags
unified_df_v2["unified_tags"] = unified_df_v2["unified_tags"].apply(to_pylist)

print(f"✅ Loaded {len(unified_df_v2):,} unique movies from V2")

# Merge V2 tags into tmdb
tmdb = tmdb.merge(unified_df_v2[["title_norm","unified_tags"]], on="title_norm", ho

# Union existing review_tags (which has V1) with V2 tags
def union_tags(a, b):
    A = to_pylist(a)
    B = to_pylist(b)
    return sorted({str(t).strip() for t in A+B})

tmdb["review_tags"] = tmdb.apply(
    lambda r: union_tags(r.get("review_tags"), r.get("unified_tags_v2")),
    axis=1
)

# Drop temporary V2 column
tmdb = tmdb.drop(columns=["unified_tags_v2"], errors="ignore")

# Validate merged results
n_with_tags = tmdb["review_tags"].apply(len).gt(0).sum()
avg_tags = tmdb["review_tags"].apply(len).mean()

print(f"✅ V2 tags merged successfully")
print(f"   Movies with tags: {n_with_tags:,}")
print(f"   Average tags per movie: {avg_tags:.2f}")

# Get total unique tags across both V1 and V2
all_tags = []
for tags in tmdb["review_tags"]:
    all_tags.extend(tags)
unique_tags = len(set(all_tags))

print(f"   Total unique tags (V1 + V2): {unique_tags}")
print("="*80)
```

```
================================================================================
📦 CONSOLIDATING V2 ZERO-SHOT TAGS
================================================================================
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
File ~\anaconda3\Lib\site-packages\pandas\core\indexes\base.py:3791, in Index.get_lo
c(self, key)
   3790 try:
-> 3791     return self._engine.get_loc(casted_key)
   3792 except KeyError as err:

File index.pyx:152, in pandas._libs.index.IndexEngine.get_loc()

File index.pyx:181, in pandas._libs.index.IndexEngine.get_loc()

File pandas\_libs\hashtable_class_helper.pxi:7080, in pandas._libs.hashtable.PyObjec
tHashTable.get_item()

File pandas\_libs\hashtable_class_helper.pxi:7088, in pandas._libs.hashtable.PyObjec
tHashTable.get_item()

KeyError: 'unified_tags'

The above exception was the direct cause of the following exception:

KeyError                                  Traceback (most recent call last)
Cell In[19], line 31
     28     except Exception: return []
     30 # Normalize V2 tags
---> 31 unified_df_v2["unified_tags"] = unified_df_v2["unified_tags"].apply(to_pylis
t)
     33 print(f"✅ Loaded {len(unified_df_v2):,} unique movies from V2")
     35 # Merge V2 tags into tmdb

File ~\anaconda3\Lib\site-packages\pandas\core\frame.py:3893, in DataFrame.__getitem
__(self, key)
   3891 if self.columns.nlevels > 1:
   3892     return self._getitem_multilevel(key)
-> 3893 indexer = self.columns.get_loc(key)
   3894 if is_integer(indexer):
   3895     indexer = [indexer]

File ~\anaconda3\Lib\site-packages\pandas\core\indexes\base.py:3798, in Index.get_lo
c(self, key)
   3793     if isinstance(casted_key, slice) or (
   3794         isinstance(casted_key, abc.Iterable)
   3795         and any(isinstance(x, slice) for x in casted_key)
   3796     ):
   3797         raise InvalidIndexError(key)
-> 3798     raise KeyError(key) from err
   3799 except TypeError:
   3800     # If we have a listlike key, _check_indexing_error will raise
   3801     #  InvalidIndexError. Otherwise we fall through and re-raise
   3802     #  the TypeError.
   3803     self._check_indexing_error(key)

KeyError: 'unified_tags'
```

In [ ]:
```
# ===== Consolidate unified checkpoint -> tmdb.review_tags (stand-alone) =====
import pandas as pd, numpy as np, ast
from pathlib import Path

master_checkpoint = ART / "zs_unified_checkpoint.parquet"
assert master_checkpoint.exists(), f"Missing {master_checkpoint}"

unified_df = pd.read_parquet(master_checkpoint)

def to_pylist(x):
    if isinstance(x, list): return x
    if isinstance(x, np.ndarray): return x.tolist()
    if x is None: return []
    if isinstance(x, str):
        try: return ast.literal_eval(x)
        except Exception: return [x]
    try: return list(x)
    except Exception: return []

unified_df["unified_tags"] = unified_df["unified_tags"].apply(to_pylist)

# robust union of existing review_tags (LDA/earlier) with unified tags
def union_tags(a, b):
    A = to_pylist(a); B = to_pylist(b)
    return sorted({str(t).strip() for t in A+B})

tmdb = tmdb.merge(unified_df[["title_norm","unified_tags"]], on="title_norm", how="
tmdb["review_tags"] = tmdb.apply(lambda r: union_tags(r.get("review_tags"), r.get("
tmdb = tmdb.drop(columns=["unified_tags"], errors="ignore")

print("Movies with review_tags:", tmdb["review_tags"].apply(len).gt(0).sum())
print("Avg tags per movie:", tmdb["review_tags"].apply(len).mean())

# (optional) persist ready-to-load dataset
tmdb.to_parquet(ART / "tmdb_with_review_tags.parquet", index=False)
print("💾 saved:", ART / "tmdb_with_review_tags.parquet")
```

In [ ]:
```
# ===== CONSOLIDATED Tag Normalization (replaces Cells 7, 8, 9) =====
# This single cell replaces the multiple attempts at normalizing review_tags

import ast
import pandas as pd
import numpy as np

def robust_to_list(x):
    """
    Convert review_tags to Python list regardless of input format.
    Handles: lists, None, NaN, numpy arrays, stringified lists, single values
    """
    # Already a proper Python list
    if isinstance(x, list):
        return x

    # None or NaN (empty)
    if x is None or (isinstance(x, float) and pd.isna(x)):
```

```
        return []

    # NumPy array (from parquet)
    if isinstance(x, np.ndarray):
        return x.tolist()

    # String representation of a list (e.g., "['tag1', 'tag2']")
    if isinstance(x, str):
        try:
            parsed = ast.literal_eval(x)
            if isinstance(parsed, (list, tuple)):
                return list(parsed)
            else:
                return [parsed]  # Single item in string form
        except (ValueError, SyntaxError):
            # If literal_eval fails, treat the whole string as a single tag
            return [x] if x.strip() else []

    # Try generic conversion to list (handles tuples, sets, etc.)
    try:
        return list(x)
    except Exception:
        return []

# Apply normalization ONCE
print("🔄 Normalizing review_tags...")
tmdb["review_tags"] = tmdb["review_tags"].apply(robust_to_list)

# Validate
n_with_tags = (tmdb["review_tags"].apply(len) > 0).sum()
avg_tags = tmdb["review_tags"].apply(len).mean()

print(f"✅ review_tags normalized successfully")
print(f"   Movies with tags: {n_with_tags:,}")
print(f"   Average tags per movie: {avg_tags:.2f}")

# Sample check
sample_tagged = tmdb[tmdb["review_tags"].apply(len) > 0].sample(min(5, n_with_tags)
print(f"\n📝 Sample tagged movies:")
for _, row in sample_tagged.iterrows():
    title = row.get("tmdb_title", "Unknown")[:40]
    tags = row["review_tags"][:3]
    print(f"  {title:40s} → {tags}")

# ===== GENRE FALLBACK TAGS for movies without reviews =====
print("\n🎯 Adding genre-based fallback tags for movies without review tags...")

def genre_fallback_tags(row):
    """Give movies without review tags some basic tags from their genres."""
    # Only apply if movie has NO review tags
    if len(row.get("review_tags", [])) > 0:
        return []

    genres = row.get("genre_names", [])
    if not genres:
        return []
```

```
    # Map genres to relevant tags
    genre_to_tags = {
        "Action": ["action", "fast-paced", "intense"],
        "Adventure": ["epic scope"],
        "Animation": ["family friendly", "feel-good"],
        "Comedy": ["feel-good", "uplifting"],
        "Crime": ["crime", "gritty", "dark"],
        "Documentary": ["documentary"],
        "Drama": ["intense", "contemplative"],
        "Family": ["family friendly", "feel-good", "uplifting"],
        "Fantasy": ["fantasy", "epic scope"],
        "History": ["historical epic", "period drama"],
        "Horror": ["horror", "dark", "suspenseful"],
        "Music": ["uplifting", "inspiring"],
        "Mystery": ["mystery", "suspenseful"],
        "Romance": ["romantic", "bittersweet"],
        "Science Fiction": ["sci-fi"],
        "Thriller": ["thriller", "suspenseful", "intense"],
        "War": ["dark", "intense", "historical epic"],
        "Western": ["western", "gritty"]
    }

    tags = []
    for g in genres:
        tags.extend(genre_to_tags.get(g, []))

    # Return max 3 unique tags
    return list(set(tags))[:3]

# Create fallback tags
tmdb["fallback_tags"] = tmdb.apply(genre_fallback_tags, axis=1)

# Union fallback tags with existing review_tags
def union_all_tags(review_tags, fallback_tags):
    A = robust_to_list(review_tags)
    B = robust_to_list(fallback_tags)
    return sorted(set(str(t).strip() for t in A + B))

tmdb["review_tags"] = tmdb.apply(
    lambda r: union_all_tags(r.get("review_tags"), r.get("fallback_tags")),
    axis=1
)

# Clean up temporary column
tmdb = tmdb.drop(columns=["fallback_tags"], errors="ignore")

# Add to end of Cell 13, BEFORE the final validation:

print("\n🔧 Adding manual override tags for famous movies...")

# Sports movies from genres
sports_genres = tmdb["genre_names"].apply(lambda g: any(x in str(g).lower() for x i
tmdb.loc[sports_genres, "review_tags"] = tmdb.loc[sports_genres, "review_tags"].app
    lambda tags: list(set(tags + ["sports"]))
)
```

```python
# Check if keywords exist
if "keywords" in tmdb.columns:
    print("   Using TMDB keywords...")
    # Extract keywords into tags
    # (would need to see your keywords column format)

print(f"✅ Override tags added")

# Re-validate
n_with_tags_after = (tmdb["review_tags"].apply(len) > 0).sum()
avg_tags_after = tmdb["review_tags"].apply(len).mean()

print(f"✅ After genre fallback:")
print(f"   Movies with tags: {n_with_tags_after:,} (was {n_with_tags:,})")
print(f"   Average tags: {avg_tags_after:.2f} (was {avg_tags:.2f})")
print(f"   Coverage: {n_with_tags_after/len(tmdb)*100:.1f}%")
```

In [ ]:

```python
# === Unify tags across sources + compute global rarity counts ===
from collections import Counter

# 1) Helper to coerce any column to clean list[str]
def _to_list(x):
    if isinstance(x, list):
        return [str(t).strip().lower() for t in x if str(t).strip()]
    if isinstance(x, str):
        # allow comma- or pipe-separated strings
        parts = [p.strip().lower() for p in x.replace("|", ",").split(",")]
        return [p for p in parts if p]
    return []

# 2) (Optional) synonym smoothing you already maintain in Master V4
THEME_SYNONYMS = {
    "kung fu": "martial arts",
    "ancient": "historical epic",
    "breakup": "lost love",
    # ← keep/add your existing mappings here
}

def _apply_synonyms(tags):
    out = []
    for t in tags:
        out.append(THEME_SYNONYMS.get(t, t))
    return out

# 3) Build a single 'all_tags' column per movie (set-union across sources)
tag_cols = []
for col in ["review_tags", "lda_themes", "zs_tags"]:
    if col in tmdb.columns:
        tag_cols.append(col)

if not tag_cols:
    print("⚠️ No tag columns found; 'all_tags' will be empty lists.")

def _unify_row(row):
    bags = []
```

```
    for c in tag_cols:
        bags.extend(_to_list(row.get(c)))
    # apply synonyms and dedupe
    return sorted(set(_apply_synonyms(bags)))

tmdb["all_tags"] = tmdb.apply(_unify_row, axis=1)

# 4) Global tag frequency (used by rarity-boost logic in tag_overlap_score)
TAG_COUNTS_GLOBAL = Counter(t for tags in tmdb["all_tags"] for t in (tags or []))

# 5) Quick visibility
print(f"✅ Tags unified. Movies with ≥1 tag: {(tmdb['all_tags'].map(bool).mean()*1
print(f"✅ Unique tags in corpus: {len(TAG_COUNTS_GLOBAL):,}")
print("   Examples:", list(TAG_COUNTS_GLOBAL.most_common(5)))
```

In [ ]:

```
# ===== FIXED movie_query_parser.py =====
# Complete THEME_SYNONYMS mapping all 104 zero-shot labels
# Better genre/character extraction

import re
from typing import Dict, List, Any

# Calculate tag rarity for boosting
from collections import Counter

all_tags = []
for tags in tmdb["review_tags"]:
    all_tags.extend(tags)

TAG_COUNTS_GLOBAL = Counter(all_tags)

print(f"📊 Tag rarity data loaded: {len(TAG_COUNTS_GLOBAL)} unique tags")
print(f"   Rarest tags (< 50 movies): {sum(1 for c in TAG_COUNTS_GLOBAL.values() if

# ===== COMPLETE THEME_SYNONYMS (maps natural language → zero-shot tags) =====
THEME_SYNONYMS = {
    # EMOTIONAL / TONAL
    "inspiring": {"inspiring", "uplifting", "hopeful"},
    "uplifting": {"inspiring", "uplifting", "hopeful"},
    "feel-good": {"feel-good", "comforting", "family friendly"},
    "feel good": {"feel-good", "comforting", "family friendly"},
    "comforting": {"comforting", "feel-good"},
    "family friendly": {"family friendly", "family values"},
    "family-friendly": {"family friendly", "family values"},
    "tragic": {"tragic", "heartbreaking", "bittersweet"},
    "heartbreaking": {"tragic", "heartbreaking"},
    "bittersweet": {"bittersweet", "lost love"},
    "lost love": {"lost love", "romantic", "bittersweet"},
    "romantic": {"romantic", "slow-burn"},
    "slow-burn": {"slow-burn", "romantic"},
    "dark": {"dark", "bleak", "gritty"},
    "bleak": {"bleak", "dark"},
    "gritty": {"gritty", "dark", "intense"},
    "mystery": {"mystery", "suspenseful"},
    "suspenseful": {"suspenseful", "thriller", "mystery"},
    "thriller": {"thriller", "suspenseful", "intense"},
```

```
    "documentary": {"documentary"},
    "coming of age": {"coming of age"},
    "coming-of-age": {"coming of age"},
    "redemption": {"redemption", "overcoming adversity"},
    "friendship": {"friendship", "found family"},
    "found family": {"found family", "friendship"},
    "biographical": {"biographical"},

    # SUBJECT / GENRE
    "mafia": {"mafia", "organized crime", "gangster", "mob"},
    "organized crime": {"organized crime", "crime", "mafia"},
    "gangster": {"gangster", "mob", "crime"},
    "gangsters": {"gangster", "mob", "crime"},  # NEW: plural
    "mob": {"mob", "mafia", "gangster"},
    "crime": {"crime", "noir", "organized crime"},
    "noir": {"noir", "crime"},
    "heist": {"heist", "crime"},
    "detective": {"detective", "mystery", "police procedural"},
    "police procedural": {"police procedural", "detective"},
    "courtroom": {"courtroom"},
    "espionage": {"espionage", "political thriller"},
    "political thriller": {"political thriller", "espionage"},

    # SCI-FI / SPECULATIVE
    "sci-fi": {"sci-fi", "science fiction"},
    "science fiction": {"science fiction", "sci-fi"},
    "cyberpunk": {"cyberpunk", "sci-fi"},
    "dystopian": {"dystopian", "post-apocalyptic"},
    "post-apocalyptic": {"post-apocalyptic", "dystopian"},
    "time travel": {"time travel", "sci-fi"},
    "space opera": {"space opera", "sci-fi", "epic scope"},
    "alien": {"alien", "sci-fi"},
    "robot ai": {"robot ai", "sci-fi"},
    "kaiju": {"kaiju", "sci-fi"},

    # HORROR / CREATURES
    "vampire": {"vampire", "horror"},
    "zombie": {"zombie", "horror"},
    "psychological horror": {"psychological horror", "horror", "dark"},
    "horror": {"horror"},

    # ACTION / MARTIAL / WESTERN
    "superhero": {"superhero", "action"},
    "martial arts": {"martial arts", "action"},
    "kung fu": {"martial arts", "action"},  # NEW!
    "karate": {"martial arts", "action"},  # NEW!
    "martial artist": {"martial arts", "action"},  # NEW!
    "western": {"western"},

    # DRAMA / ART
    "period drama": {"period drama"},
    "historical epic": {"historical epic", "epic scope"},
    "ancient": {"historical epic", "period drama"},  # NEW!
    "ancient times": {"historical epic", "period drama"},  # NEW!
    "ancient history": {"historical epic", "period drama"},  # NEW!
    "medieval": {"historical epic", "period drama"},  # NEW!
```

```
    "surreal": {"surreal", "absurdist"},
    "absurdist": {"absurdist", "surreal"},
    "existential": {"existential", "cerebral"},
    "arthouse": {"arthouse", "cerebral"},
    "satire": {"satire", "parody"},
    "parody": {"parody", "satire"},

    # FANTASY
    "fantasy": {"fantasy"},
    "sword and sorcery": {"sword and sorcery", "fantasy"},
    "myth and legend": {"myth and legend", "fantasy"},

    # CHARACTER TYPES
    "strong female lead": {"strong female lead", "female protagonist"},
    "female lead": {"strong female lead", "female protagonist"},
    "female protagonist": {"female protagonist", "strong female lead"},
    "male protagonist": {"male protagonist"},
    "ensemble cast": {"ensemble cast"},
    "anti-hero": {"anti-hero"},
    "underdog": {"underdog story"},
    "underdog story": {"underdog story"},
    "mentor": {"mentor relationship"},
    "father-son": {"father-son relationship"},
    "father": {"father-son relationship"},  # NEW!
    "dad": {"father-son relationship"},  # NEW!
    "son": {"father-son relationship"},  # NEW!
    "boy and his father": {"father-son relationship"},  # NEW!
    "mother-daughter": {"mother-daughter relationship"},
    "mother": {"mother-daughter relationship"},  # NEW!
    "unlikely friendship": {"unlikely friendship"},

    # RELATIONSHIP DYNAMICS
    "forbidden love": {"forbidden love", "romantic"},
    "unrequited love": {"unrequited love", "lost love"},
    "toxic relationship": {"toxic relationship"},
    "family drama": {"family drama"},
    "generational conflict": {"generational conflict", "family drama"},
    "class struggle": {"class struggle"},
    "cultural clash": {"cultural clash"},

    # EMOTIONAL JOURNEY / THEMES
    "revenge": {"revenge", "justice vs revenge"},
    "betrayal": {"betrayal"},
    "sacrifice": {"sacrifice"},
    "survival": {"survival"},
    "overcoming adversity": {"overcoming adversity", "redemption"},
    "self-discovery": {"self-discovery", "identity crisis"},
    "identity crisis": {"identity crisis", "self-discovery"},
    "moral dilemma": {"moral dilemma"},
    "loss and grief": {"loss and grief"},
    "grief": {"loss and grief"},
    "hope": {"hope", "inspiring"},
    "justice": {"justice vs revenge"},
    "loneliness": {"loneliness"},
    "power and corruption": {"power and corruption"},
    "corruption": {"power and corruption"},
```

```
    # PACING / ATMOSPHERE
    "fast-paced": {"fast-paced", "intense"},
    "fast paced": {"fast-paced", "intense"},
    "intense": {"intense", "fast-paced"},
    "contemplative": {"contemplative", "cerebral"},
    "whimsical": {"whimsical"},
    "atmospheric": {"atmospheric"},
    "cerebral": {"cerebral", "contemplative"},

    # SETTING / SCOPE
    "small-town": {"small-town", "isolated setting"},
    "small town": {"small-town", "isolated setting"},
    "isolated": {"isolated setting"},
    "road trip": {"road trip"},
    "epic": {"epic scope"},
    "intimate": {"intimate story"},

    # BREAKUPS & RELATIONSHIPS (NEW!)
    "breakup": {"lost love", "heartbreaking", "romantic"},
    "broke up": {"lost love", "heartbreaking", "romantic"},
    "broken heart": {"lost love", "heartbreaking"},
    "break up": {"lost love", "heartbreaking", "romantic"},

    # SPORTS (TEMPORARY MAPPING - will improve with V2)
    "sports": {"action"},  # Temp until V2 adds sports tags
    "sport": {"action"},
    "football": {"action"},
    "basketball": {"action"},
    "baseball": {"action"},
    "soccer": {"action"},
    "boxing": {"action"},
    "athletic": {"action"},

    # GENOCIDE / HOLOCAUST (MAPPING TO EXISTING TAGS)
    "genocide": {"dark", "tragic", "historical epic"},
    "holocaust": {"dark", "tragic", "historical epic"},
    "war crimes": {"dark", "tragic"},
}

# Avoid false actor matches
BAD_ACTOR_WORDS = {
    "dark", "psychological", "family", "inspiring", "romantic", "sad", "happy",
    "thriller", "drama", "comedy", "action", "horror", "documentary"
}

def query_theme_set(q: str) -> set:
    """Extract theme tags from query using THEME_SYNONYMS."""
    ql = q.lower()
    out = set()
    for kw, tags in THEME_SYNONYMS.items():
        if kw in ql:
            out |= tags
    return out

# Emotional intent detection
```

```python
SAD_WORDS = {"sad", "grief", "melancholy", "heartbroken", "lost love", "lonely", "d
HAPPY_WORDS = {"feel good", "feel-good", "happy", "uplifting", "inspiring", "hopefu

def wants_sad(q: str) -> bool:
    ql = q.lower()
    return any(kw in ql for kw in SAD_WORDS)

def wants_happy(q: str) -> bool:
    ql = q.lower()
    return any(kw in ql for kw in HAPPY_WORDS)

def parse_query_safe(q: str):
    """
    Parse query with BAD_ACTOR_WORDS filter.
    Returns filters dict with: year_min, year_max, actor, genres, etc.
    """
    f = parse_query(q)
    if "actor" in f:
        a = str(f["actor"]).lower()
        if any(w in a for w in BAD_ACTOR_WORDS):
            f.pop("actor", None)
    return f

# Original parse_query function (placeholder - use your existing one)
def parse_query(q: str) -> Dict[str, Any]:
    """Parse query - extract year, actor, genres."""
    filters = {}

    # Extract decades (90s, 1990s, etc.) - FIXED VERSION
    decade_match = re.search(r"(\d{2,4})s", q)
    if decade_match:
        decade_str = decade_match.group(1)
        try:
            if len(decade_str) == 2:
                # Handle "80s", "90s" → "1980", "1990"
                decade = int("19" + decade_str)
            else:
                decade = int(decade_str)

            filters["year_min"] = decade
            filters["year_max"] = decade + 9
        except ValueError:
            # If conversion fails, skip decade filtering
            pass

    # Extract explicit years
    year_match = re.search(r"(19|20)\d{2}", q)
    if year_match:
        try:
            year = int(year_match.group(0))
            filters["year_min"] = year
            filters["year_max"] = year
        except ValueError:
            pass

    # Extract actor names (simple pattern)
```

```python
    # Look for common patterns like "X movies" or "with X"
    actor_patterns = [
        r"([\w\s]+?)\s+movies",  # "Bill Murray movies"
        r"with\s+([\w\s]+?)(?:\s+from|\s+in|\s*$)",  # "with Jennifer Lopez"
        r"starring\s+([\w\s]+?)(?:\s+from|\s+in|\s*$)",  # "starring Tom Hanks"
    ]

    for pattern in actor_patterns:
        match = re.search(pattern, q, re.IGNORECASE)
        if match:
            potential_actor = match.group(1).strip()
            # Basic validation: 2-4 words, capitalized
            words = potential_actor.split()
            if 1 <= len(words) <= 4:
                filters["actor"] = potential_actor.title()
                break

    # Extract genres
    genre_keywords = {
        "thriller": "Thriller",
        "drama": "Drama",
        "comedy": "Comedy",
        "action": "Action",
        "horror": "Horror",
        "sci-fi": "Science Fiction",
        "science fiction": "Science Fiction",
        "romance": "Romance",
        "documentary": "Documentary",
        "animation": "Animation",
        "fantasy": "Fantasy",
        "mystery": "Mystery",
        "crime": "Crime",
        "adventure": "Adventure",
        "war": "War",
        "western": "Western"
    }

    ql = q.lower()
    genres = []
    for keyword, genre_name in genre_keywords.items():
        if keyword in ql:
            genres.append(genre_name)

    if genres:
        filters["genres"] = genres

    return filters


def filter_by_metadata(df, filters):
    """Apply metadata filters to dataframe."""
    result = df.copy()

    # Year filters
    if "year_min" in filters:
        result = result[result["year"] >= filters["year_min"]]
```

```
    if "year_max" in filters:
        result = result[result["year"] <= filters["year_max"]]

    # ACTOR FILTERING (NEW IMPLEMENTATION)
    if "actor" in filters:
        actor_name = filters["actor"].lower()

        # Check if 'cast' column exists
        if "cast" in result.columns:
            # Filter by cast column (handles list/string formats)
            def has_actor(cast_data):
                if pd.isna(cast_data):
                    return False

                # Convert to string and check
                cast_str = str(cast_data).lower()
                return actor_name in cast_str

            result = result[result["cast"].apply(has_actor)]
        else:
            print(f"⚠️ WARNING: 'cast' column not found, cannot filter by actor '{a

    # Genre filters
    if "genres" in filters:
        req_genres = set(g.lower() for g in filters["genres"])

        def has_genre(genre_list):
            if not genre_list:
                return False
            movie_genres = set(g.lower() for g in genre_list)
            return bool(movie_genres & req_genres)

        result = result[result["genre_names"].apply(has_genre)]

    return result

# Scoring utilities
def tag_overlap_score(tags, q_themes: set):
    """Score with RARE TAG BOOSTING - rewards specific/niche tags."""
    tags = set(map(str, (tags or [])))

    # If no query themes, use a neutral score that doesn't dominate
    if not q_themes:
        return 0.7  # Keep this neutral

    if not tags:
        return 0.5

    inter = tags & q_themes
    if not inter:
        return 0.5

    # Calculate base score
    base_score = 0.5 + 0.5 * (len(inter) / max(1, len(tags | q_themes)))

    # BOOST RARE TAGS (the magic fix!)
```

```
    rarity_boost = 0.0
    for tag in inter:
        tag_count = TAG_COUNTS_GLOBAL.get(tag, 0)

        if tag_count < 50:  # Ultra-rare (sports, martial arts)
            rarity_boost += 0.4
        elif tag_count < 200:  # Rare (gangster, western)
            rarity_boost += 0.25
        elif tag_count < 1000:  # Uncommon (sci-fi, fantasy)
            rarity_boost += 0.1
        # Common tags (intense, inspiring) get no boost

    final_score = min(1.0, base_score + rarity_boost)
    return final_score

def soft_genre_score(genres, req_genres):
    """Score based on genre overlap."""
    if not req_genres:
        return 0.7
    g = set(map(str.lower, (genres or [])))
    q = set(map(str.lower, req_genres))
    if not g:
        return 0.7
    inter = len(g & q)
    uni = len(g | q)
    return 0.6 + 0.4 * (inter / max(1, uni))  # 0.6–1.0

def sentiment_match_score(row, q: str):
    """
    Expanded sentiment blending:
    - For 'sad/lost love' intents, use sentiment_sad
    - For 'feel-good/uplifting' intents, use sentiment_happy
    - Otherwise neutral 0.7
    """
    ql = q.lower()
    if any(w in ql for w in SAD_WORDS):
        val = row.get("sentiment_sad", None)
    elif any(w in ql for w in HAPPY_WORDS):
        val = row.get("sentiment_happy", None)
    else:
        return 0.7

    try:
        if val is None or (hasattr(val, '__iter__') and len(val) == 0):
            return 0.7
        import pandas as pd
        if pd.isna(val):
            return 0.7
        return float(0.3 + 0.7 * max(0.0, min(1.0, float(val))))
    except Exception:
        return 0.7
```

In [ ]:

```
# ===== CompleteRecommender v3.1 - Split Actor Fallback + Bug Fixes =====

import numpy as np
import pandas as pd
```

```python
import ast
from sentence_transformers import SentenceTransformer

class CompleteRecommender:
    def __init__(self, df, svd, ncf, user_map, movie_map,
                 faiss_idx, emb, emb_keys, key2idx, key_set, title_mode=True):
        self.df = df
        self.svd = svd
        self.ncf = ncf
        self.user_to_idx = user_map
        self.movie_to_idx = movie_map
        self.faiss = faiss_idx
        self.emb = emb
        self.emb_keys = emb_keys
        self.key2idx = key2idx
        self.key_set = key_set
        self.title_mode = title_mode
        self.encoder = SentenceTransformer('sentence-transformers/all-MiniLM-L6-v2'

    @staticmethod
    def _content(qv, key, key2idx, emb):
        idx = key2idx.get(key)
        if idx is None:
            return 3.0
        sim = float(np.dot(qv[0], emb[idx]))
        return float(np.clip(1 + 4*((sim+1)/2), 1, 5))

    def _cf(self, user_raw, movie_raw):
        if user_raw is None or movie_raw is None:
            return 2.5

        try:
            svd_pred = self.svd.predict(str(user_raw), str(movie_raw)).est
            svd_scaled = float(np.clip(svd_pred, 1, 5))
        except Exception:
            svd_scaled = 3.0

        uid = self.user_to_idx.get(str(user_raw))
        mid = self.movie_to_idx.get(str(movie_raw))

        if uid is not None and mid is not None:
            try:
                p01 = float(self.ncf.predict([np.array([uid]), np.array([mid])],
                                             verbose=0).reshape(-1)[0])
                ncf_scaled = float(np.clip(1 + 4*p01, 1, 5))
            except Exception:
                ncf_scaled = 3.0
        else:
            ncf_scaled = 3.0

        return 0.5*svd_scaled + 0.5*ncf_scaled

    def _detect_query_type(self, filters, q_themes, query):
        """Detect query type for adaptive weighting."""
        has_factual = any(k in filters for k in ["year_min", "year_max", "actor", "
        has_semantic = len(q_themes) > 0 or wants_sad(query) or wants_happy(query)
```

```python
        if has_factual and not has_semantic:
            return "factual"
        elif has_semantic and not has_factual:
            return "semantic"
        else:
            return "mixed"

    def _get_adaptive_weights(self, query_type, sad_flag, user_id, query):
        """Return adaptive weights based on query type. FIX: Added query parameter.
        if query_type == "factual":
            if user_id:
                return (0.30, 0.50, 0.05, 0.10, 0.05)
            else:
                return (0.40, 0.30, 0.10, 0.15, 0.05)

        elif query_type == "semantic":
            if sad_flag or wants_happy(query):
                return (0.35, 0.25, 0.20, 0.05, 0.15)
            else:
                return (0.40, 0.25, 0.25, 0.05, 0.05)

        else:  # mixed
            if sad_flag:
                return (0.30, 0.30, 0.15, 0.10, 0.15)
            else:
                return (0.35, 0.35, 0.15, 0.10, 0.05)

    def _score_candidates(self, cand_df, q_themes: set, filters: dict):
        """
        Score candidates with tag/theme relevance (rarity-boosted) first, then qual
        then genre & sentiment nudges. Expects:
          - cand_df['all_tags'] list[str]
          - cand_df['genre_names'] list[str]
          - cand_df['weighted_rating'], ['vote_count'], ['vote_average'] (optional)
          - cand_df['sentiment_score'] (optional)
        """
        from movie_query_parser import (
            tag_overlap_score, soft_genre_score, sentiment_match_score,
            wants_sad, wants_happy
        )

        # ensure required columns exist
        if "all_tags" not in cand_df.columns:
            cand_df["all_tags"] = [[] for _ in range(len(cand_df))]
        if "genre_names" not in cand_df.columns:
            cand_df["genre_names"] = [[] for _ in range(len(cand_df))]

        q_themes = set(q_themes or [])
        want_sad = wants_sad(q_themes)
        want_happy = wants_happy(q_themes)

        scored = []
        for _, r in cand_df.iterrows():
            # 1️⃣ theme relevance (rarity-boosted via TAG_COUNTS_GLOBAL)
            theme = float(tag_overlap_score(r.get("all_tags", []), q_themes))
```

```
        # 2️⃣ quality signals
        wr = float(r.get("weighted_rating", r.get("vote_average", 0.0)) or 0.0)
        votes = min(1.0, float(r.get("vote_count", 0) or 0.0) / 10000.0)

        # 3️⃣ genre & sentiment nudges
        genre = float(
            soft_genre_score(r.get("genre_names", []), filters.get("genres", []
        ) if filters and "genres" in filters else 0.0

        sent = float(r.get("sentiment_score", 0.0))
        sent_adj = float(
            sentiment_match_score(sent, want_sad, want_happy)
        ) if q_themes else 0.0

        # 4️⃣ weighted blend — tag relevance dominates
        final = (
            0.55 * theme +
            0.15 * wr +
            0.05 * votes +
            0.10 * genre +
            0.05 * sent_adj +
            0.10 * 0.0    # reserved
        )

        scored.append({
            "title": r.get("tmdb_title"),
            "year": r.get("year"),
            "genres": r.get("genre_names", []),
            "themes": r.get("all_tags", []),
            "rating": r.get("vote_average", 0.0),
            "weighted_rating": r.get("weighted_rating", 0.0),
            "content": 0.0, "cf": 0.0,
            "theme": theme,
            "genre_match": genre,
            "sentiment": sent_adj,
            "score": float(final)
        })

    scored.sort(key=lambda x: x["score"], reverse=True)
    return scored




def recommend(self, query, user_id=None, n=10):
    import os
    import pandas as pd_local
    from pathlib import Path
    from movie_query_parser import (
        parse_query, parse_query_safe,
        filter_by_metadata,
        query_theme_set, wants_sad, wants_happy,
        tag_overlap_score, soft_genre_score, sentiment_match_score
    )
```

```
        # 1) Parse & encode
        filters = parse_query_safe(query)
        qv = self.encoder.encode([query]).astype(np.float32)
        qv /= np.linalg.norm(qv, axis=1, keepdims=True) + 1e-12

        q_themes = query_theme_set(query)
        req_genres = filters.get("genres", [])
        sad_flag = wants_sad(query)

        query_type = self._detect_query_type(filters, q_themes, query)

        # 2) FAISS search
        D, I = self.faiss.search(qv, min(3000, self.faiss.ntotal))
        faiss_keys = [self.emb_keys[i] for i in I[0] if i != -1]
        faiss_set = set(faiss_keys)

        # 3) SPLIT ACTOR FALLBACK: Try full query, then split if actor query fails
        has_actor = "actor" in filters
        cand = None
        fallback_level = 0
        actor_fallback_results = None

        # Level 0: Strict intersection (all filters + FAISS)
        meta_df = filter_by_metadata(self.df, filters).copy()
        meta_df["__key__"] = meta_df["title_norm"].astype(str)
        cand = meta_df[meta_df["__key__"].isin(faiss_set)].copy()

        # SPLIT FALLBACK: If actor query returns <5 results, provide two result set
        if has_actor and len(cand) < 5:
            actor_name = filters.get("actor")

            # Result Set A: Drop actor, keep genre/decade/themes
            filters_no_actor = {k: v for k, v in filters.items() if k != "actor"}
            meta_no_actor = filter_by_metadata(self.df, filters_no_actor).copy()
            meta_no_actor["__key__"] = meta_no_actor["title_norm"].astype(str)
            cand_no_actor = meta_no_actor[meta_no_actor["__key__"].isin(faiss_set)]

            # Result Set B: Keep actor, drop other constraints
            filters_actor_only = {"actor": actor_name}
            if "year_min" in filters:
                filters_actor_only["year_min"] = filters["year_min"]
            if "year_max" in filters:
                filters_actor_only["year_max"] = filters["year_max"]

            meta_actor_only = filter_by_metadata(self.df, filters_actor_only).copy(
            meta_actor_only["__key__"] = meta_actor_only["title_norm"].astype(str)
            cand_actor_only = meta_actor_only.head(500).copy()

            if len(cand_no_actor) > 0:
                cand = cand_no_actor.head(1200).copy()
                fallback_level = "split_primary"

                # Score actor fallback separately
                if len(cand_actor_only) > 0:
                    content_w, cf_w, tag_w, genre_w, sentiment_w = self._get_adapti
```

```
                    "factual", sad_flag, user_id, query  # Actor-only is factua
                )

                actor_fallback_results = self._score_candidates(
                    cand_actor_only.head(500), qv, user_id, query, q_themes, []
                    0.30, 0.50, 0.05, 0.05, 0.10, "split_actor", "factual"
                )
            else:
                cand = cand_actor_only
                fallback_level = "actor_only"

        # Continue with normal fallback if still empty
        if len(cand) == 0 and len(meta_df) > 0:
            cand = meta_df.head(500).copy()
            fallback_level = 1

        if len(cand) == 0:
            temp_df = self.df.copy()
            temp_df["__key__"] = temp_df["title_norm"].astype(str)
            cand = temp_df[temp_df["__key__"].isin(faiss_set)].head(500).copy()
            fallback_level = 2

        if len(cand) == 0:
            cand = self.df.nlargest(500, "weighted_rating").copy()
            cand["__key__"] = cand["title_norm"].astype(str)
            fallback_level = 3

        if len(cand) == 0:
            return []

        cand = cand.head(1200).copy()

        # Get adaptive weights
        content_w, cf_w, tag_w, genre_w, sentiment_w = self._get_adaptive_weights(
            query_type, sad_flag, user_id, query
        )

        # 4) Score primary candidates
        out = self._score_candidates(
            cand, qv, user_id, query, q_themes, req_genres,
            content_w, cf_w, tag_w, genre_w, sentiment_w, fallback_level, query_typ
        )

        # 5) Re-ranker (optional)
        ART = Path(r"C:\Users\kylek\artifacts").resolve()
        RE_RANKER_PATH = ART / "re_ranker_lgb.txt"

        def apply_reranker(results):
            if os.path.exists(RE_RANKER_PATH) and len(results) > 0:
                import lightgbm as lgb
                booster = lgb.Booster(model_file=str(RE_RANKER_PATH))
                rX = pd_local.DataFrame([{
                    "content": r["content"],
                    "cf": r["cf"],
                    "tag_score": r.get("theme", 0.7),
                    "weighted_rating": (r.get("weighted_rating", 5.0) or 5.0) / 10.
```

```
                } for r in results])

                rerank_scores = booster.predict(rX)
                for r, s in zip(results, rerank_scores):
                    r["rerank"] = float(s)

                results.sort(key=lambda x: x.get("rerank", -1.0), reverse=True)
            else:
                results.sort(key=lambda x: (x["score"], x.get("weighted_rating", 0.
            return results

        out = apply_reranker(out)

        # Apply reranker to actor fallback too
        if actor_fallback_results:
            actor_fallback_results = apply_reranker(actor_fallback_results)

        # Return format: If split happened, return dict with both sets
        if actor_fallback_results:
            return {
                "primary": out[:n],
                "actor_fallback": actor_fallback_results[:n],
                "split_query": True,
                "actor_name": filters.get("actor")
            }
        else:
            return out[:n]
```

In [ ]:

```
# ===== COMBINED: Instantiation + Testing (All-in-One Cell) =====

# ============================================================================
# PART 1: INSTANTIATE RECOMMENDER
# ============================================================================

print("🚀 Instantiating CompleteRecommender...")

recommender = CompleteRecommender(
    df=tmdb,
    svd=svd_model,
    ncf=ncf_model,
    user_map=user_to_idx,
    movie_map=movie_to_idx,
    faiss_idx=faiss_index,
    emb=emb,
    emb_keys=emb_keys,      # ← CRITICAL for v3.1
    key2idx=key2idx,
    key_set=key_set,
    title_mode=TITLE_MODE
)

print("✅ Recommender ready (Master v4 - v3.1)")
print(f"   • Loaded {len(tmdb):,} movies")
print(f"   • {len(user_to_idx):,} users, {len(movie_to_idx):,} CF IDs")

# ============================================================================
# PART 2: PRETTY_PRINT FUNCTION
```

```python
# ============================================================================

def pretty_print(results):
    """Updated to handle split actor fallback format."""

    # Check if split format (actor query with fallback)
    if isinstance(results, dict) and "split_query" in results:
        actor_name = results.get("actor_name", "actor")
        primary = results.get("primary", [])
        actor_fallback = results.get("actor_fallback", [])

        print(f"\n{'='*80}")
        print(f"📌 SPLIT RESULTS (exact match not found)")
        print(f"{'='*80}")

        print(f"\n🎬 PRIMARY: Matching genre/themes/decade (without {actor_name}):"
        print(f"{'-'*80}")
        if not primary:
            print("   No results")
        else:
            for i, r in enumerate(primary, 1):
                g = ", ".join(r["genres"][:3]) if isinstance(r["genres"], list) els
                print(f"{i:2d}. {r['title'][:55]:55s} ({r['year']}) ⭐{r['rating']
                if g:
                    print(f"     🎭 {g}")
                if r.get("themes"):
                    print(f"     🏷️  {', '.join(map(str, r['themes'][:3]))}")
                print(f"     📊 Content={r['content']:.2f} CF={r['cf']:.2f} Theme={
                print()

        print(f"\n👤 ACTOR FALLBACK: All {actor_name} movies from specified time:"
        print(f"{'-'*80}")
        if not actor_fallback:
            print("   No results")
        else:
            for i, r in enumerate(actor_fallback, 1):
                g = ", ".join(r["genres"][:3]) if isinstance(r["genres"], list) els
                print(f"{i:2d}. {r['title'][:55]:55s} ({r['year']}) ⭐{r['rating']
                if g:
                    print(f"     🎭 {g}")
                if r.get("themes"):
                    print(f"     🏷️  {', '.join(map(str, r['themes'][:3]))}")
                print(f"     📊 Content={r['content']:.2f} CF={r['cf']:.2f} Theme={
                print()

        return

    # Normal format (list of movies)
    if not results:
        print("❌ No results")
        return

    for i, r in enumerate(results, 1):
        g = ", ".join(r["genres"][:3]) if isinstance(r["genres"], list) else ""
        print(f"{i:2d}. {r['title'][:55]:55s} ({r['year']}) ⭐{r['rating']:.1f} | 
        if g:
```

```
            print(f"    🎭 {g}")
        if r.get("themes"):
            print(f"    🏷️  {', '.join(map(str, r['themes'][:3]))}")
        print(f"    📊 Content={r['content']:.2f} CF={r['cf']:.2f} Theme={r.get('tl
        print()

# ============================================================================
# PART 3: TEST QUERIES
# ============================================================================

print("\n" + "="*80)
print("🧪 RUNNING TEST QUERIES")
print("="*80)

# Test query list
queries = [
    "Bill Murray movies from the 90s rated above 7",
    "like Breaking Bad but with a female lead",
    "an inspiring, family-friendly drama",
    "dark psychological thriller from the 2010s",
    "I'm sad and want a movie about lost love",
    "dark psychological thriller with Jennifer Lopez from the 90s"  # Tests split f
]

# Run each query
for q in queries:
    print(f"\n--- Query: {q} ---")
    try:
        results = recommender.recommend(q, n=10)
        pretty_print(results)
    except Exception as e:
        print(f"❌ ERROR: {e}")
        import traceback
        traceback.print_exc()

print("\n" + "="*80)
print("✅ All tests complete!")
print("="*80)
```

In [ ]:

```
# Run this in a new cell - show me the output
print("=== DIAGNOSTIC CHECK ===")
print(f"1. THEME_SYNONYMS exists: {'THEME_SYNONYMS' in dir()}")
if 'THEME_SYNONYMS' in dir():
    print(f"   Length: {len(THEME_SYNONYMS)}")
else:
    print("   ❌ NOT LOADED")

print(f"\n2. emb_keys exists: {'emb_keys' in dir()}")
if 'emb_keys' in dir():
    print(f"   Type: {type(emb_keys)}")
    print(f"   Length: {len(emb_keys)}")
else:
    print("   ❌ NOT CREATED")

print(f"\n3. CompleteRecommender class:")
print(f"   Has _get_adaptive_weights: {hasattr(recommender, '_get_adaptive_weights'
```

```
print(f"   Has _score_candidates: {hasattr(recommender, '_score_candidates')}")

# Check parser function
from movie_query_parser import parse_query_safe
test_parse = parse_query_safe("Bill Murray movies from the 90s")
print(f"\n4. Parser test: {test_parse}")

# Check theme extraction
from movie_query_parser import query_theme_set
test_themes = query_theme_set("dark psychological thriller")
print(f"\n5. Theme extraction test: {test_themes}")
```

In [ ]:

```
# ===== Cell D: Gradio Interface (OPTIONAL) =====
# Place AFTER Cell C if you want the interactive UI

import gradio as gr

def recommend_ui(query, user_id, top_n):
    if not query.strip():
        return "Please enter a query"

    user = user_id.strip() if user_id.strip() else None
    results = recommender.recommend(query, user_id=user, n=int(top_n))

    if not results:
        return "No results found"

    output = []
    for i, r in enumerate(results, 1):
        genres = ", ".join(r["genres"][:3]) if r["genres"] else "N/A"
        themes = ", ".join(map(str, r["themes"][:3])) if r["themes"] else "N/A"
        output.append(
            f"{i}. **{r['title']}** ({r['year']})\n"
            f"    ⭐ {r['rating']:.1f} | WR: {r['weighted_rating']:.2f}\n"
            f"    🎭 {genres}\n"
            f"    🏷️ {themes}\n"
        )
    return "\n".join(output)

demo = gr.Interface(
    fn=recommend_ui,
    inputs=[
        gr.Textbox(label="Query", placeholder="Type a query (natural language), opt
        gr.Textbox(label="User ID (optional)", placeholder=""),
        gr.Slider(1, 20, value=10, step=1, label="Top N")
    ],
    outputs=gr.Textbox(label="Results"),
    title="Hybrid Movie Recommender",
    description="Type a query (natural language), optionally add a training user ID
    allow_flagging="never"
)

demo.launch(inline=True, height=520, show_error=True)
```

In [ ]:
```python
# Check what tags are actually being used
from collections import Counter

# Get all tags across all movies
all_tags = []
for tags in tmdb["review_tags"]:
    all_tags.extend(tags)

tag_counts = Counter(all_tags)

print("🔝 Top 30 most common tags:")
for tag, count in tag_counts.most_common(30):
    print(f"  {tag:30s} → {count:,} movies")

print(f"\n📊 Total unique tags: {len(tag_counts)}")

# Check specific tags we care about
print("\n🎯 Specific tag coverage:")
important_tags = [
    "martial arts", "sports", "kung fu", "boxing",
    "ancient rome", "ancient greece", "medieval", "historical epic",
    "gangster", "mob", "mafia", "organized crime",
    "father-son relationship", "mother-daughter relationship"
]

for tag in important_tags:
    count = tag_counts.get(tag, 0)
    print(f"  {tag:30s} → {count:,} movies")

# Check if Enter the Dragon exists
print("\n🥋 Checking for famous martial arts movies:")
martial_arts_movies = ["enter the dragon", "ip man", "crouching tiger hidden dragon
                       "the raid", "ong bak", "kung fu panda"]

for title in martial_arts_movies:
    matches = tmdb[tmdb["title_norm"].str.contains(title, na=False)]
    if len(matches) > 0:
        movie = matches.iloc[0]
        print(f"  {movie['tmdb_title']:40s} → Tags: {movie['review_tags']}")
    else:
        print(f"  '{title}' → NOT IN DATASET")
```

In [ ]:
```python
# Check if famous movies are tagged correctly
famous_checks = {
    "goodfellas": "gangster",
    "the godfather": "gangster",
    "gladiator": "historical epic",
    "300": "historical epic",
    "rocky": "sports",
    "hoosiers": "sports",
    "enter the dragon": "martial arts"
}

for title_search, expected_tag in famous_checks.items():
    matches = tmdb[tmdb["title_norm"].str.contains(title_search, na=False)]
```

```
    if len(matches) > 0:
        movie = matches.iloc[0]
        has_tag = expected_tag in movie["review_tags"]
        symbol = "✅" if has_tag else "❌"
        print(f"{symbol} {movie['tmdb_title']:40s} → {expected_tag:20s} → {movie['r
```

In [ ]:
```
print("\n=== AMERICAN PIE CHECK ===")
american_pie = tmdb[tmdb['tmdb_title'].str.contains('American Pie', case=False, na=
print(f"Movies matching 'American Pie': {len(american_pie)}")
if len(american_pie) > 0:
    for _, row in american_pie.iterrows():
        print(f"  - {row['tmdb_title']} ({row.get('year')})")
        print(f"    Cast: {row.get('cast', 'NO CAST DATA')[:200]}")
```

In [ ]:
```
print("=== FAISS INDEX CHECK ===")

# Check if American Pie is in the embeddings
test_titles = ["american pie", "american pie 1999", "american pie 2001"]

for title in test_titles:
    normalized = norm_title(title)  # Use your norm_title function
    print(f"\nSearching for: '{title}' → normalized: '{normalized}'")

    if normalized in key_set:
        idx = key2idx[normalized]
        print(f"  ✅ FOUND in FAISS at index {idx}")
    else:
        print(f"  ❌ NOT FOUND in FAISS index")

# Check what's actually in emb_keys around "american"
print(f"\n📋 Sample of titles starting with 'american' in FAISS:")
american_titles = [k for k in emb_keys[:10000] if k.startswith('american')]
for t in american_titles[:10]:
    print(f"  - {t}")

print(f"\nTotal 'american' titles in FAISS: {len(american_titles)}")
```

In [ ]:
```
print("=== DATA STRUCTURE MISMATCH CHECK ===\n")

print(f"1. key_set size: {len(key_set):,}")
print(f"2. key2idx size: {len(key2idx):,}")
print(f"3. emb_keys size: {len(emb_keys):,}")
print(f"4. FAISS index size: {faiss_index.ntotal:,}")

print(f"\n5. Are sizes consistent?")
print(f"   key_set == key2idx: {len(key_set) == len(key2idx)}")
print(f"   key2idx == emb_keys: {len(key2idx) == len(emb_keys)}")
print(f"   emb_keys == FAISS: {len(emb_keys) == faiss_index.ntotal}")

# Check what emb_keys actually contains
print(f"\n6. First 10 emb_keys:")
for i, key in enumerate(emb_keys[:10]):
    print(f"   [{i}] {key}")

# Check if emb_keys has the same entry at index 14564
```

```python
if len(emb_keys) > 14564:
    print(f"\n7. emb_keys[14564] = '{emb_keys[14564]}'")
else:
    print(f"\n7. ❌ emb_keys only has {len(emb_keys)} entries, can't access index 1

# Check tmdb vs key_set overlap
tmdb_titles_normalized = tmdb['tmdb_title'].apply(norm_title)
overlap = sum(1 for t in tmdb_titles_normalized if t in key_set)
print(f"\n8. tmdb titles in key_set: {overlap:,} / {len(tmdb):,} ({100*overlap/len(
```

In [ ]: